



ЛЕКЦИЯ 10

Подготовка признаков и построение воспроизводимых конвейеров машинного обучения

Машинное обучение и безопасность систем

Магистратура НИУ ВШЭ (МИЭМ) · ОП «Информационная безопасность и технологии ИИ»

Автор: Силаев Ю. В. · Время изучения: ≈ 70-75 минут



Эта лекция за одну минуту

2 / 56

Часть 1 · Введение

Ключевой тезис и маршрут лекции

Качество ML-решения часто определяется не выбором алгоритма, а тем, как подготовлены признаки: как обработаны пропуски и категории, как предотвращены утечки на этапе препроцессинга и как организован конвейер эксперимента.

Главный тезис лекции: признаки и конвейер обработки данных – это часть модели. Если они построены с ошибками или утечками, итоговая оценка качества теряет смысл. Поэтому конвейер должен быть воспроизводимым артефактом, а не последовательностью разовых преобразований в notebook.

МАРШРУТ ЛЕКЦИИ

- Типы признаков
- Пропуски, выбросы, масштабирование
- Категориальные признаки
- Время и агрегаты
- Pipeline и ColumnTransformer



Учебные результаты – что вы сможете делать после изучения лекции

- ✓ Вы сможете различать числовые, категориальные, бинарные, временные и агрегированные признаки и выбирать способ их обработки;
- ✓ Вы сможете корректно обрабатывать пропуски, выбросы и категориальные переменные;
- ✓ Вы сможете технически реализовывать отдельную обработку типов признаков в едином конвейере (Pipeline, ColumnTransformer);
- ✓ Вы сможете встраивать препроцессинг в конвейер так, чтобы он обучался только на train и применялся к val/test без утечек;
- ✓ Вы сможете строить и сериализовать воспроизводимый конвейер обработки данных и инференса;
- ✓ Вы сможете выявлять признаки-утечки и учитывать в ИБ источник, время и механизм формирования данных.



Предварительные требования и связь с предыдущими лекциями

Перед этой лекцией вы должны быть знакомы с материалом:

- L05 – протоколы валидации, train/test split и типология утечек данных (data leakage);
- L02 – эксплоративный анализ: как обнаруживаются проблемы данных (пропуски, выбросы, аномалии распределений);
- L06 / L08 – линейные модели и деревья/ансамбли как модели, которые встраиваются в конвейер.

СВЯЗЬ С L05

Типология утечек данных введена в L05.

В этой лекции нет ни одного нового типа утечки. Здесь – только техника защиты от уже известных типов: как технически устроить препроцессинг, чтобы он не «подсматривал» в test.



План лекции

5 / 56

Часть 1 · Введение

Семь частей · ориентировочное время самостоятельного изучения

Часть	Содержание	Слайды	Время
Часть 1	Введение	Слайды 1-5	~6 мин
Часть 2	Мотивация: зачем нужен воспроизводимый конвейер	Слайды 6-9	~5 мин
Часть 3	Основное содержание: признаки, обработка, Pipeline	Слайды 10-41	~40 мин
Часть 4	Связки с практикой ИБ	Слайды 42-49	~10 мин
Часть 5	Итоги	Слайды 50-53	~4 мин
Часть 6	Источники для углубления	Слайды 54-55	~3 мин
Часть 7	Глоссарий	Слайд 56	~2 мин



ЧАСТЬ 2

Зачем инженеру воспроизводимый конвейер

Что ломается, когда препроцессинг – это набор разрозненных ячеек notebook



Где на самом деле создаётся и теряется качество ML-решения

В учебных задачах кажется, что качество модели определяется выбором алгоритма: «возьмём бустинг вместо логистической регрессии – станет лучше». На практике в задачах ИБ разрыв между алгоритмами часто меньше, чем разрыв между хорошо и плохо подготовленными признаками на одном и том же алгоритме.

Подготовка данных – обработка пропусков, кодирование категорий, масштабирование, конструирование временных и агрегированных признаков – определяет, что вообще «видит» модель. Ошибка здесь не исправляется выбором более сложного алгоритма: если в признаках утечка из будущего, любая модель покажет завышенное качество на валидации и провалится в эксплуатации.

ГДЕ РЕШАЕТСЯ КАЧЕСТВО

выбор алгоритма

малый вклад в результат

подготовка признаков

большой вклад и главный источник ошибок

защита от утечек

определяет, можно ли верить оценке



Реальный сценарий из работы SOC: модель «работала в ноутбуке»



И Б - К Е Й С

Аналитик обучил детектор подозрительных сетевых соединений в Jupyter: посчитал среднее и дисперсию для масштабирования, заполнил пропуски медианой, закодировал протоколы – всё на полном датасете, затем разбил на train/test. На валидации ROC-AUC ≈ 0.98 .

При переносе в продакшен модель получает события по одному, медианы и средние «из будущего» недоступны, порядок преобразований восстановить не удаётся. Реальное качество оказалось заметно ниже валидационного, а воспроизвести препроцессинг на новых данных невозможно.

Корень проблемы: препроцессинг существовал как набор разрозненных ячеек, обученных на полном датасете, а не как единый объект, обучаемый только на train и применяемый дальше без изменений.



Три типовых провала, которые повторяются в реальных ML-системах



ЧАСТАЯ ОШИБКА

- Масштабирование, импутация и кодирование выполняются на всём датасете до разбиения на train/test.
- В признаки попадает информация из будущего или результат расследования – фактически ответ.
- Модели сравниваются на разных наборах признаков без зафиксированного протокола преобразований.



ПОЧЕМУ ЭТО НЕВЕРНО

Оценка качества завышена: модель «подсмотрела» статистику теста, и валидация не предсказывает поведение в эксплуатации.

Инференс невоспроизводим: на новых данных нет тех же статистик и порядка шагов.

Сравнение моделей бессмысленно: разница в метриках объясняется признаками, а не моделями.



Б Л О К А

Признаки: что это и какими бывают

Признак – это решение инженера, а не свойство, заданное самими данными



Что мы вообще подаём на вход модели

ОПРЕДЕЛЕНИЕ

Признак (feature) – измеримая характеристика объекта, представленная числом или категорией. Набор признаков объекта – это вектор $\mathbf{x} \in \mathbb{R}^d$, который и есть всё, что модель «знает» об объекте.

Ключевая мысль: модель не видит объект напрямую – она видит только его проекцию в признаки. Один и тот же сетевой пакет можно описать пятью числами или пятьюдесятью; можно сохранить порт как число, как категорию или как флаг «известный/неизвестный». Это инженерное решение, и оно определяет, что модель в принципе способна различить. Если важная для задачи информация не попала в признаки, никакой алгоритм её не восстановит.



Один и тот же датасет проходит через весь курс – узнавайте его



CIC-IDS-2018: ЧТО ЭТО

Публичный датасет сетевого трафика (Canadian Institute for Cybersecurity, 2018).

- ≈ 16 млн flow-записей за 10 дней наблюдения;
- около 80 признаков на каждое соединение;
- 14 типов атак (DoS, DDoS, Brute Force, Bot, Infiltration и др.) плюс класс Benign;
- сильный дисбаланс классов: Benign-трафика во много раз больше атак.

ЗАЧЕМ ОН В ЭТОЙ ЛЕКЦИИ

80 признаков CIC-IDS-2018 – естественная смесь типов: числовые «скоростные» (байты/с, пакеты/с), категориальные флаги протокола, бинарные индикаторы.

Именно такая разнотипность делает датасет идеальным примером для ColumnTransformer – к нему мы вернёмся в блоке E.



Тип признака определяет способ его обработки в конвейере

Тип	Что это	Пример в ИБ
Числовые	Непрерывные величины	длительность соединения, объём трафика в байтах
Категориальные	Конечный набор значений без порядка	протокол (TCP/UDP/ICMP), сервис, доменная зона
Бинарные	Два значения – частный случай категориальных	флаг SYN, признак «порт из списка известных»
Счётчики / частоты	Число событий или их доля за период	число пакетов, доля ошибочных запросов в сессии
Временные	Производные от метки времени	час суток, день недели, интервал между событиями
Агрегаты по окнам	Сводные статистики за временное окно	среднее число соединений с хоста за 5 минут



Как 80 признаков распадаются на группы – задел под ColumnTransformer



ЧИСЛОВЫЕ «СКОРОСТНЫЕ»

Flow Bytes/s, Flow Packets/s, Flow Duration, средний размер пакета

→ *масштабирование*
(*StandardScaler*)



КАТЕГОРИАЛЬНЫЕ ФЛАГИ

Protocol (TCP/UDP/ICMP), Destination Port как категория сервиса

→ *one-hot encoding*



БИНАРНЫЕ ИНДИКАТОРЫ

наличие флага SYN/ACK/FIN, признак «порт из известных»

→ *передаются без изменений*

Каждая группа требует своей обработки – но всё это должно жить в одном конвейере. Как именно – в блоке E.



- ✓ Модель видит не объект, а его проекцию в признаки. Что не попало в признаки – для модели не существует.
- ✓ Признак – инженерное решение: одно и то же свойство можно закодировать числом, категорией или флагом.
- ✓ Основные типы: числовые, категориальные, бинарные, счётчики/частоты, временные, агрегаты по окнам.
- ✓ Тип признака диктует обработку: числовые масштабируют, категориальные кодируют, бинарные часто передают как есть.
- ✓ ИБ-данные (CIC-IDS-2018) почти всегда разнотипны – это прямая мотивация для ColumnTransformer (блок E).



Б Л О К В

Очистка и приведение числовых признаков

Пропуски, выбросы, масштабирование – и почему всё это учится только на train



Обнаружение проблем – в L02 (EDA); здесь – как их обрабатывать в конвейере

Пропуск (missing value) – отсутствие значения признака у объекта.

Базовые стратегии обработки: **удаление** объектов или признаков с пропусками; **импутация** – заполнение значением (медианой для числовых, модой для категориальных); **индикатор пропуска** – добавление бинарного признака «значение отсутствовало».

Критично для конвейера: значение для импутации (медиана, мода) вычисляется только на train и затем применяется к val/test. Медиана, посчитанная по всему датасету, – это утечка статистики теста (тип утечки по preprocessing – как в L05).

КОГДА ЧТО ПРИМЕНЯТЬ

Удаление

мало пропусков, они случайны

Импутация

пропусков заметно, признак нужен

Индикатор

сам факт пропуска может быть сигналом



Частое заблуждение: пропуск считают только технической проблемой



ЧАСТАЯ ОШИБКА

«Пропуск – это просто отсутствующее значение, его нужно поскорее заполнить медианой и забыть». Заполняя пропуск, инженер молча уничтожает информацию о том, что значение отсутствовало – а в ИБ это часто и есть сигнал.

Почему в ИБ это важно: отсутствие поля часто несёт смысл. Пустой User-Agent, отсутствующий referrer, незаполненное поле геолокации, «дырки» в логах в момент атаки – всё это может коррелировать с вредоносной активностью сильнее, чем само значение. Поэтому индикатор пропуска (бинарный признак «было пусто») нередко полезнее аккуратной импутации: он сохраняет сигнал вместо того, чтобы его маскировать.



Аномально большие или малые значения, выбивающиеся из распределения

Выброс (outlier) – значение, далеко отстоящее от основной массы наблюдений. Обнаружение – на уровне идеи: правило межквартильного размаха (за пределами $Q1 - 1.5 \cdot IQR$ и $Q3 + 1.5 \cdot IQR$), отклонение на несколько стандартных отклонений, либо просто визуально по распределению (L02).

Способы обработки без удаления данных:

- winsorization – обрезка экстремальных значений до выбранного перцентиля (например, до 1-го и 99-го);
- устойчивые преобразования – логарифмирование тяжёлых правых хвостов ($\log(1+x)$), которое сжимает шкалу;
- робастное масштабирование – по медиане и IQR вместо среднего и дисперсии.



Самая дорогая ошибка блока: бездумное удаление выбросов



ЧАСТАЯ ОШИБКА

«Выбросы – это шум и грязь в данных, их надо удалить перед обучением».

В обычном ML такое иногда оправдано. Но в задачах безопасности именно редкое, аномальное событие чаще всего и есть то, что мы ищем.



ПОЧЕМУ ЭТО НЕВЕРНО

DDoS – это выброс по числу пакетов. Эксфильтрация – выброс по объёму исходящего трафика. Брутфорс – выброс по частоте неудачных входов.

Удаляя выбросы «для чистоты», вы удаляете ровно те объекты, которые должны детектировать. Модель учится на «спокойном» трафике и слепнет к атакам.



Приведение признаков к сопоставимым шкалам

ФОРМУЛА

Standardization: $z = (x - \mu) / \sigma$

Normalization: $x' = (x - x_{\min}) / (x_{\max} - x_{\min})$

μ , σ – среднее и стандартное отклонение признака; $x_{\min}$, $x_{\max}$ – его минимум и максимум. Standardization даёт нулевое среднее и единичную дисперсию; normalization сжимает значения в отрезок [0, 1].

μ и σ оцениваются ТОЛЬКО на train. Среднее и дисперсия, посчитанные по всему датасету, – классическая утечка по preprocessing (как в L05).



Зависит от семейства модели – это не универсальный обязательный шаг

НУЖНО МАСШТАБИРОВАТЬ

- Линейные модели и логистическая регрессия – иначе признак с большим масштабом доминирует.
- Методы на расстояниях: kNN, k-means, SVM с RBF – расстояние искажается несопоставимыми шкалами.
- Нейронные сети – стабилизирует и ускоряет обучение (понадобится в L13).
- PCA и методы на дисперсии – иначе «крупный» признак захватывает главные компоненты.

МОЖНО НЕ МАСШТАБИРОВАТЬ

- Решающие деревья – делят по порогам, монотонные преобразования им безразличны.
- Случайный лес и градиентный бустинг (L08) – наследуют это свойство деревьев.
- Правила на исходных единицах, где важна интерпретируемость абсолютных значений.



- ✓ Пропуски обрабатывают удалением, импутацией или индикатором; сам факт пропуска в ИБ часто является сигналом.
- ✓ Выбросы не обязательно удалять: winsorization, логарифмирование и робастное масштабирование сохраняют данные.
- ✓ В ИБ выброс часто и есть атака (DDoS, эксфильтрация, брутфорс) – удаление выбросов ослепляет детектор.
- ✓ Масштабирование нужно линейным, дистанционным и нейросетевым моделям; деревьям и бустингу – нет.
- ✓ Все параметры обработки (μ , σ , медианы, границы winsorization) оцениваются только на train – иначе утечка (L05).



БЛОК С

Кодирование категориальных признаков

Как превратить категории в числа – и не занести при этом утечку



Два базовых способа представить категорию числом

ONE-HOT ENCODING

Каждое значение категории становится отдельным бинарным признаком (0/1). Протокол {TCP, UDP, ICMP} → три столбца, ровно один из которых равен 1.

Подходит для номинальных признаков без порядка. Не навязывает ложных отношений между значениями, но увеличивает размерность.

ORDINAL ENCODING

Каждой категории присваивается целое число: {low, medium, high} → {0, 1, 2}. Один столбец вместо многих.

Корректно только если у категорий есть естественный порядок. Для номинальных признаков (протокол, страна) ordinal навязывает ложный порядок: модель решит, что ICMP «больше» TCP.



Когда у категориального признака тысячи или миллионы значений

Кардинальность – число различных значений категориального признака. В ИБ она часто огромна: **IP-адреса, порты, домены, имена процессов, user-agent-строки** могут иметь десятки тысяч уникальных значений. One-hot для такого признака порождает десятки тысяч столбцов – матрица становится разреженной, обучение замедляется, переобучение растёт.

Базовые приёмы снижения: группировка редких значений в категорию «прочее»; переход к осмысленным агрегатам (не сам порт, а «порт из списка известных сервисов»); хэширование категорий в фиксированное число корзин.

ПОРЯДОК КАРДИНАЛЬНОСТИ

Протокол	~3
Порт	~65 000
Домен	$10^4 - 10^6$
IP-адрес	$10^5 +$
User-Agent	$10^3 - 10^5$



Соблазнительный приём для высокой кардинальности – и как он протекает

Идея target encoding: заменить категорию средним значением целевой переменной для этой категории (например, доля атак среди соединений на данный порт). Один столбец вместо тысяч – заманчиво при высокой кардинальности.



ЧАСТАЯ ОШИБКА

Студент считает target encoding по всему датасету, а затем разбивает на train/test. В кодировке каждой категории уже «зашит» ответ, в том числе по тестовым объектам.



ПОЧЕМУ ЭТО НЕВЕРНО

Это утечка по target (как в L05): признак содержит информацию о метке. Защита – считать кодировку только на train, а лучше внутри каждого фолда CV (out-of-fold), со сглаживанием для редких категорий.



- ✓ One-hot – для номинальных признаков без порядка; не навязывает ложных отношений, но увеличивает размерность.
- ✓ Ordinal – только при настоящем порядке категорий; иначе модель видит ложное «больше/меньше».
- ✓ В ИБ кардинальность часто огромна (IP, порты, домены) – нужны группировка редких значений, осмысленные агрегаты, хэширование.
- ✓ Target encoding мощен, но опасен: посчитанный на полном датасете, он даёт утечку по target (L05).
- ✓ Защита для target encoding – считать кодировку только на train, в идеале out-of-fold внутри кросс-валидации.



Б Л О К D

Время, агрегаты и отбор признаков

Конструирование признаков из времени и событий – на прикладном уровне



Из одной метки времени можно извлечь много полезных признаков

Сырая метка времени (timestamp) сама по себе плохой признак: модель не понимает её как момент в цикле. Полезнее производные признаки:

- календарные – час суток, день недели, выходной/будний, месяц: ловят суточную и недельную сезонность активности;
- интервалы – время с прошлого события того же пользователя или хоста;
- лаги – значение признака на несколько шагов назад;
- оконные агрегаты – сводные статистики за временное окно (см. следующий слайд).

ГРАНИЦА ТЕМЫ

Здесь – прикладной уровень: какие признаки времени бывают и как не занести утечку.

Полноценный временной анализ – sessionization, time-based split, rolling validation, concept drift – в L12.



Самая коварная утечка блока – агрегат, заглянувший вперёд

Оконный агрегат – статистика события за интервал времени: «сколько соединений с этого хоста за последние 5 минут», «средний объём трафика за час». Мощный класс признаков для логов и событий.



ЧАСТАЯ ОШИБКА

Окно считают симметрично вокруг события («±5 минут») или по всему ряду сразу. В признак для момента t попадают данные из $t+1$, $t+2$ – из будущего относительно предсказания.



ПОЧЕМУ ЭТО НЕВЕРНО

Это временная утечка (как в L05): в эксплуатации будущего нет. Правило – окно строится только из прошлого: $[t-\Delta, t]$. Агрегаты считаются причинно, в порядке времени, без заглядывания вперёд.



Как из потока событий собрать признаки сессии и SOC-алерта



ПРИЗНАКИ СЕССИИ

число действий, доля ошибок входа, нетипичное время активности, география входа, длительность сессии



ПРИЗНАКИ SOC-АЛЕРТА

источник, тип сработавшего правила, частота повторения, история похожих событий за период



ПРИЗНАКИ ХОСТА

число соединений за окно, число уникальных адресатов, доля исходящего трафика

Общий принцип: одно «сырое» событие беднее, чем его контекст. Агрегаты по окну превращают отдельный лог в поведенческий профиль – но строятся строго из прошлого (предыдущий слайд). Детальная техника оконных признаков для логов – в L12.



Зачем убирать признаки – и где проходит граница темы

Больше признаков – не всегда лучше. Шумовые, дублирующие и почти константные признаки удлиняют обучение, повышают риск переобучения и затрудняют интерпретацию. Базовая интуиция отбора:

- убрать признаки с нулевой или почти нулевой дисперсией (одно значение почти всюду);
- убрать сильно коррелированные дубли, оставив один представитель группы;
- отсеять признаки с пренебрежимо малой связью с целевой переменной;
- опираться на важность признаков из модели (impurity importance из L08, подробнее – L09).



ГРАНИЦА ТЕМЫ

Сложные методы отбора и causal feature selection – вне рамок курса. Здесь – только базовая инженерная гигиена признаков.



Б Л О К Е

Конвейер как программный артефакт

Pipeline и ColumnTransformer: единственный код-блок курса на sklearn



Препроцессинг и модель как единый обучаемый объект

ОПРЕДЕЛЕНИЕ

Pipeline – объект, инкапсулирующий всю последовательность преобразований данных и финальную модель в одну сущность. `pipe.fit(X_train, y_train)` обучает всю цепочку, `pipe.predict(X_new)` прогоняет новые данные через те же шаги.

Главная идея: препроцессинг – это часть модели, а не подготовительный шаг «до» неё. Каждый шаг конвейера обучается (`fit`) и применяется (`transform`). Pipeline гарантирует, что обучение всех шагов произошло на train, а к val/test применяются уже зафиксированные параметры – без возможности случайно «подсмотреть» в тест.



sklearn.pipeline.Pipeline – последовательность (имя, преобразователь)

```
# импорты опущены: Pipeline, SimpleImputer,  
# StandardScaler, LogisticRegression  
  
pipe = Pipeline([  
    ('impute', SimpleImputer(strategy='median')),  
    ('scale', StandardScaler()),  
    ('clf', LogisticRegression(max_iter=1000)),  
])  
  
pipe.fit(X_train, y_train)      # учится только на train  
y_pred = pipe.predict(X_test)  # те же шаги, без утечки
```

Импутация, масштабирование и модель – один объект. Медиана и параметры StandardScaler выучиваются на train внутри fit и переиспользуются в predict.



Как Pipeline защищает от утечки

37 / 56

Часть 3 · Pipeline

Техническая реализация принципа «обучить препроцессинг только на train» (L05)



Параметры текут только слева направо: train → параметры → test. Обратного пути нет.

Без Pipeline инженер легко вызывает `scaler.fit(X)` на всём X до split – и параметры впитывают статистику теста. Pipeline делает такую ошибку структурно невозможной: `fit` видит только train, `transform` применяет выученное к test. Это и есть техническая защита от утечки по preprocessing, концептуально введённой в L05.



Разные преобразования для разных групп столбцов – в одном объекте

ОПРЕДЕЛЕНИЕ

ColumnTransformer – объект, применяющий свой преобразователь к каждой группе столбцов и склеивающий результат. Числовые → масштабирование, категориальные → one-hot, бинарные → без изменений – параллельно и согласованно, внутри одного fit/transform.

Зачем это нужно: реальные данные разнотипны (блок A). Без ColumnTransformer пришлось бы обрабатывать группы столбцов вручную, по отдельности, и вручную же склеивать – теряя контроль над тем, что на чём обучалось. ColumnTransformer вкладывается внутрь Pipeline как один из шагов, и вся защита от утечки сохраняется автоматически.



Разнотипные признаки сетевого трафика – в одном конвейере

```
# импорты: ColumnTransformer, StandardScaler, OneHotEncoder

numeric = ['Flow Bytes/s', 'Flow Packets/s', 'Flow Duration']
categ   = ['Protocol', 'Dst Port Group']
binary  = ['flag_SYN', 'is_known_port']

prep = ColumnTransformer([
    ('num', StandardScaler(),      numeric),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categ),
    ('bin', 'passthrough',        binary),
])
pipe = Pipeline([('prep', prep), ('clf', clf)])
```

Три группы признаков CIC-IDS-2018 (блок A) обрабатываются согласованно. Весь препроцессинг – один шаг 'prep' внутри Pipeline.



Капкан: fit_transform вне Pipeline

40 / 56

Часть 3 · Pipeline

Самая частая практическая ошибка из паспорта лекции



ЧАСТАЯ ОШИБКА

```
X_all = scaler.fit_transform(X)
```

```
X_tr, X_te = split(X_all)
```

Отдельные fit_transform по ячейкам notebook, до или мимо split. Параметры выучены на полном датасете.



ПОЧЕМУ ЭТО НЕВЕРНО

Теряется контроль над тем, что обучалось на train, а что – на полном датасете. Это утечка по preprocessing (L05), которую глазами в notebook не видно.

Решение – поместить ВСЕ преобразования в Pipeline. Тогда fit физически не может коснуться test, а порядок шагов и параметры фиксируются в одном объекте.



- ✓ Pipeline инкапсулирует препроцессинг и модель в один объект: препроцессинг – это часть модели.
- ✓ `fit` обучает все шаги только на `train`; `transform` применяет выученные параметры к `val/test` – утечка структурно невозможна.
- ✓ `ColumnTransformer` обрабатывает разнотипные группы столбцов согласованно и вкладывается внутрь `Pipeline` как один шаг.
- ✓ На CIC-IDS-2018: числовые → `scaler`, категориальные → `one-hot`, бинарные → `passthrough` в одном конвейере.
- ✓ Отдельные `fit_transform` вне `Pipeline` – главный источник скрытой утечки по preprocessing (L05).



ЧАСТЬ 4

Признаки и конвейеры в задачах ИБ

Что отличает feature engineering в безопасности от обычного ML



Самый опасный класс признаков в ИБ-задачах

ОПРЕДЕЛЕНИЕ

Признак-утечка – признак, который прямо или косвенно содержит ответ: информацию о целевой переменной, недоступную в момент реального предсказания. Модель на нём показывает блестящие метрики на валидации и бесполезна в эксплуатации.

Почему ИБ особенно уязвима: разметка атак часто появляется в результате расследования – постфактум. Поля, заполненные аналитиком или системой реагирования, физически не существуют в момент, когда модель должна принять решение. Если такой признак попал в обучение, мы предсказываем атаку по следам её обнаружения, а не по самой атаке.



Признаки, появляющиеся постфактум

44 / 56

Часть 4 · Практика ИБ

Конкретные примеры утечки target в SOC-данных



ИБ-КЕЙС

В таблице инцидентов есть поля «категория угрозы», «назначенный аналитик», «время закрытия тикета», «вердикт». Студент берёт их как признаки и получает почти идеальную классификацию.

Но все эти поля заполняются ПОСЛЕ того, как инцидент признан атакой. В момент, когда событие только поступило, их нет. Модель выучила не атаку, а факт, что по событию уже провели расследование.

Правило проверки: для каждого признака спросите – был ли он доступен в момент предсказания? Если значение появляется только после события или после реакции на него, это утечка, и признак нужно убрать.



Часть признаков отражает сбор данных, а не поведение атакующего



ЧАСТАЯ ОШИБКА

Модель отлично отделяет атаки – но по признакам вроде конкретного сенсора-источника, диапазона TTL, версии формата лога или временного окна сбора.

Так бывает, когда атаки и Benign в датасете собирались разной инфраструктурой или в разное время.



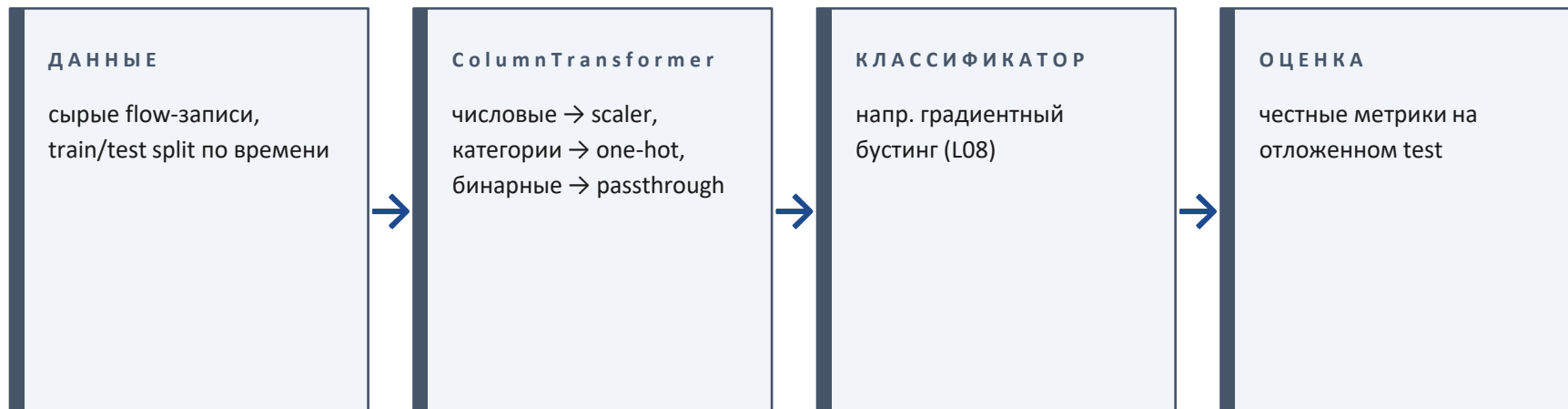
ПОЧЕМУ ЭТО НЕВЕРНО

Модель выучила артефакт стенда, а не атаку. На реальном трафике, где инфраструктура одна для всех классов, такой признак исчезает – и качество рушится.

Это связано с проблемами данных из L02 и напрямую ведёт к теме сдвига распределений в эксплуатации (L17).



Сборка всего, что мы разобрали, в один воспроизводимый объект



Всё – один объект Pipeline. Препроцессинг обучается только на train-части (разбитой по времени, чтобы не было временной утечки), затем тот же объект целиком применяется к test. Сравнение разных классификаторов идёт при идентичном препроцессинге – честный протокол.



Один и тот же обученный объект – на train и в продакшене

Воспроизводимость инференса означает: на новых данных применяется ровно тот конвейер, что был обучен, со всеми выученными параметрами и порядком шагов. Для этого обученный Pipeline сохраняется как единый артефакт (сериализуется) и загружается в продакшене.

Это закрывает ровно ту проблему, с которой началась лекция (мотивация): в notebook препроцессинг жил как набор ячеек и не переносился. Сериализованный Pipeline переносит и препроцессинг, и модель как одно целое.

ЧТО ХРАНИТ АРТЕФАКТ

- порядок всех шагов препроцессинга;
- выученные параметры (μ , σ , медианы, словари категорий);
- обученную модель;
- версии – чтобы инференс был детерминирован.



В ИБ распределения признаков быстро меняются – почему это важно уже сейчас

В безопасности распределение признаков нестабильно: обновляется инфраструктура, меняется поведение пользователей и тактики атакующих. Признак, информативный сегодня, может стать бесполезным или вводящим в заблуждение через месяц.

Минимальная гигиена уже на этапе построения признаков:

- сравнивать распределения признаков между train и test (резкое расхождение – повод насторожиться);
- избегать признаков, жёстко завязанных на конкретный период или версию инфраструктуры;
- помнить, что модель и признаки придётся периодически переобучать.

В ПЕРЁД → L17

Систематически дрейф, covariate/label shift и стресс-тесты на сдвиг данных разбираются в L17.



Минимальный чек-лист признаков до того, как нажать fit



ЧЕК-ЛИСТ ПРИЗНАКОВ

- | | |
|---|---|
| ✓ Нет ли среди признаков таких, что доступны только после события? (признак-утечка) | ✓ Проверена ли доля пропусков и кардинальность категорий? |
| ✓ Сравнены ли распределения признаков между train и test? | ✓ Нет ли признаков, отражающих инфраструктуру сбора, а не объект? |
| ✓ Обучается ли весь препроцессинг внутри Pipeline только на train? | ✓ Зафиксированы ли порядок шагов и параметры (сериализация)? |



ЧАСТЬ 5

Итоги лекции

Что унести с собой из L10



- ✓ Признаки и конвейер обработки данных – это часть модели, а не подготовка «до» неё.
- ✓ Весь препроцессинг обучается только на train; параметры применяются к val/test без изменений – иначе утечка (L05).
- ✓ Pipeline инкапсулирует препроцессинг и модель в один объект и делает утечку по preprocessing структурно невозможной.
- ✓ ColumnTransformer обрабатывает разнотипные признаки согласованно внутри одного конвейера.
- ✓ В ИБ пропуск и выброс часто являются сигналом, а не мусором; выброс может быть самой атакой.
- ✓ Признак-утечка (доступный только постфактум) убивает оценку качества – его нужно выявлять и убирать.
- ✓ Конвейер – воспроизводимый сериализуемый артефакт, а не последовательность ячеек в notebook.



Границы применимости того, что разобрано в лекции

ПРИМЕНЯТЬ ВСЕГДА

- в любом табличном ML-эксперименте: оборачивать препроцессинг и модель в Pipeline;
- при разнотипных признаках – ColumnTransformer;
- при подготовке модели к переносу в продакшен – сериализация конвейера;
- при любой оценке качества – препроцессинг только на train.

ЗА РАМКАМИ ЭТОЙ ЛЕКЦИИ

- промышленные feature stores и управление признаками в масштабе;
- автоматизированный feature engineering как отдельная область;
- сложные методы отбора и causal feature selection;
- продвинутая обработка временных рядов – это L12.



Связь с другими темами курса

53 / 56

Часть 5 · Итоги

Куда двигаться дальше для самостоятельной навигации

Лекция	Тема	Связь с L10
L05	Типология leakage и схемы валидации	концепция, которую L10 реализует технически
L02	EDA: обнаружение проблем данных	то, что обнаружено там, исправляется в pipeline
L08 / L09	Деревья и интерпретируемость	модели в конце конвейера; важность признаков
L12	Логи и временная структура	оконные признаки и time-based split – детально
L13	Основы нейросетей	pipeline понадобится; масштабирование критично
L15	Тексты и журналы	текстовый препроцессинг как часть pipeline
L17 / L19	Сдвиг данных; аудит данных	дрейф признаков; аудит конвейера и артефактов



Минимум для понимания темы

Hastie, Tibshirani, Friedman. The Elements of Statistical Learning. Гл. 7 «Model Assessment and Selection» – почему оценка качества требует честного отделения train от всего, что влияет на модель (включая препроцессинг).

Bishop. Pattern Recognition and Machine Learning. Гл. 1 (introduction) – роль признаков и предобработки; общая постановка задачи обучения.

Документация scikit-learn. Разделы «Pipelines and composite estimators» (`sklearn.pipeline.Pipeline`, `sklearn.compose.ColumnTransformer`) и «Preprocessing data» – основной практический справочник по теме лекции.



Для тех, кто хочет глубже, и практические материалы

УГЛУБИТЬСЯ

- Kuhn, Johnson. Feature Engineering and Selection – системный разбор конструирования и отбора признаков.
- Материалы по data leakage в прикладном ML – углубление концепции из L05 на практических кейсах.
- Документация по сериализации моделей (joblib) – практика переноса конвейера в инференс.

ПРАКТИКА

- Тьюториал scikit-learn «Column Transformer with Mixed Types» – пошаговая сборка ColumnTransformer.
- Тьюториал «Common pitfalls and recommended practices» (sklearn) – раздел про data leakage и его предотвращение.
- CIC-IDS-2018 как площадка: построить полный Pipeline для разнотипных признаков самостоятельно.



Признак (feature)	Измеримая характеристика объекта (число или категория).
Pipeline	Объект, инкапсулирующий препроцессинг и модель в одну сущность.
ColumnTransformer	Раздельная обработка групп столбцов внутри одного конвейера.
Импутация	Заполнение пропусков значением (медиана, мода).
Индикатор пропуска	Бинарный признак «значение отсутствовало».
Winsorization	Обрезка экстремальных значений до выбранного перцентиля.

Standardization	Масштабирование к нулевому среднему и единичной дисперсии.
One-hot encoding	Кодирование категории набором бинарных столбцов.
Target encoding	Замена категории средним target; рискует утечкой.
Признак-утечка	Признак, содержащий ответ или данные из будущего.
Оконный агрегат	Статистика события за интервал времени (только из прошлого).
Сериализация	Сохранение обученного конвейера как единого артефакта.